

5. ProfileRegistry.sol

Purpose: Tracks Elev8 (long-term) and Gener8 (quarterly) scores per user, used for DAO access, rewards, and staking tiers.

Solidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/access/AccessControl.sol";

/**
 * @title ProfileRegistry
 * @dev Stores Elev8 and Gener8 ratings per user for the Utiligram ecosystem.
 */
contract ProfileRegistry is AccessControl {
    bytes32 public constant EVALUATOR_ROLE =
        keccak256("EVALUATOR_ROLE");

    struct Profile {
        uint256 elev8Score; // long-term score (e.g. 0–10000 scale)
        uint256 gener8Score; // resets yearly
        uint8 gener8Tier; // 0 = Bronze, 1 = Silver, 2 = Gold, 3 = Platinum
        uint256 lastReset; // timestamp of last Gener8 reset
    }

    mapping(address => Profile) public profiles;

    event ProfileUpdated(address indexed user, uint256 elev8, uint256 gener8, uint8 tier);
    event Gener8Reset(address indexed user, uint256 newScore, uint8 newTier);

    constructor() {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    }

    function setProfile(address user, uint256 elev8, uint256 gener8, uint8 tier) external
        onlyRole(EVALUATOR_ROLE) {
        profiles[user].elev8Score = elev8;
        profiles[user].gener8Score = gener8;
        profiles[user].gener8Tier = tier;
        emit ProfileUpdated(user, elev8, gener8, tier);
    }

    function resetGener8(address user, uint256 newScore, uint8 newTier)
        external onlyRole(EVALUATOR_ROLE) {
        profiles[user].gener8Score = newScore;
        profiles[user].gener8Tier = newTier;
        profiles[user].lastReset = block.timestamp;
        emit Gener8Reset(user, newScore, newTier);
    }

    function getProfile(address user)
        external view returns (Profile memory) {
        return profiles[user];
    }

    function
    isEligibleForElev8Boost(address user)
        external view returns (bool) {
        return block.timestamp >= profiles[user].lastReset + 365 days;
    }
}
```

